

Chapter 6

Working Classes

cc2e.com/0665

Contents

- 6.1 Class Foundations: Abstract Data Types (ADTs): page 126
- 6.2 Good Class Interfaces: page 133
- 6.3 Design and Implementation Issues: page 143
- 6.4 Reasons to Create a Class: page 152
- 6.5 Language-Specific Issues: page 156
- 6.6 Beyond Classes: Packages: page 156

Related Topics

- Design in construction: Chapter 5
- Software architecture: Section 3.5
- High-quality routines: Chapter 7
- The Pseudocode Programming Process: Chapter 9
- Refactoring: Chapter 24

In the dawn of computing, programmers thought about programming in terms of statements. Throughout the 1970s and 1980s, programmers began thinking about programs in terms of routines. In the twenty-first century, programmers think about programming in terms of classes.



KEY POINT

A class is a collection of data and routines that share a cohesive, well-defined responsibility. A class might also be a collection of routines that provides a cohesive set of services even if no common data is involved. A key to being an effective programmer is maximizing the portion of a program that you can safely ignore while working on any one section of code. Classes are the primary tool for accomplishing that objective.

This chapter contains a distillation of advice in creating high-quality classes. If you're still warming up to object-oriented concepts, this chapter might be too advanced. Make sure you've read Chapter 5, "Design in Construction." Then start with Section 6.1, "Class Foundations: Abstract Data Types (ADTs)," and ease your way into the remaining sections. If you're already familiar with class basics, you might skim Section 6.1 and then dive into the discussion of class interfaces in Section 6.2. The "Additional Resources" section at the end of this chapter contains pointers to introductory reading, advanced reading, and programming-language-specific resources.

6.1 Class Foundations: Abstract Data Types (ADTs)

An abstract data type is a collection of data and operations that work on that data. The operations both describe the data to the rest of the program and allow the rest of the program to change the data. The word “data” in “abstract data type” is used loosely. An ADT might be a graphics window with all the operations that affect it, a file and file operations, an insurance-rates table and the operations on it, or something else.

Cross-Reference Thinking about ADTs first and classes second is an example of programming *into* a language vs. programming *in* one. See Section 4.3, “Your Location on the Technology Wave,” and Section 34.4, “Program into Your Language, Not in It.”

Understanding ADTs is essential to understanding object-oriented programming. Without understanding ADTs, programmers create classes that are “classes” in name only—in reality, they are little more than convenient carrying cases for loosely related collections of data and routines. With an understanding of ADTs, programmers can create classes that are easier to implement initially and easier to modify over time.

Traditionally, programming books wax mathematical when they arrive at the topic of abstract data types. They tend to make statements like “One can think of an abstract data type as a mathematical model with a collection of operations defined on it.” Such books make it seem as if you’d never actually use an abstract data type except as a sleep aid.

Such dry explanations of abstract data types completely miss the point. Abstract data types are exciting because you can use them to manipulate real-world entities rather than low-level, implementation entities. Instead of inserting a node into a linked list, you can add a cell to a spreadsheet, a new type of window to a list of window types, or another passenger car to a train simulation. Tap into the power of being able to work in the problem domain rather than at the low-level implementation domain!

Example of the Need for an ADT

To get things started, here’s an example of a case in which an ADT would be useful. We’ll get to the details after we have an example to talk about.

Suppose you’re writing a program to control text output to the screen using a variety of typefaces, point sizes, and font attributes (such as bold and italic). Part of the program manipulates the text’s fonts. If you use an ADT, you’ll have a group of font routines bundled with the data—the typeface names, point sizes, and font attributes—they operate on. The collection of font routines and data is an ADT.

If you’re not using ADTs, you’ll take an ad hoc approach to manipulating fonts. For example, if you need to change to a 12-point font size, which happens to be 16 pixels high, you’ll have code like this:

```
currentFont.size = 16
```

If you've built up a collection of library routines, the code might be slightly more readable:

```
currentFont.size = PointsToPixels( 12 )
```

Or you could provide a more specific name for the attribute, something like

```
currentFont.sizeInPixels = PointsToPixels( 12 )
```

But what you can't do is have both *currentFont.sizeInPixels* and *currentFont.sizeInPoints*, because, if both the data members are in play, *currentFont* won't have any way to know which of the two it should use. And if you change sizes in several places in the program, you'll have similar lines spread throughout your program.

If you need to set a font to bold, you might have code like this that uses a logical *or* and a hexadecimal constant *0x02*:

```
currentFont.attribute = currentFont.attribute or 0x02
```

If you're lucky, you'll have something cleaner than that, but the best you'll get with an ad hoc approach is something like this:

```
currentFont.attribute = currentFont.attribute or BOLD
```

Or maybe something like this:

```
currentFont.bold = True
```

As with the font size, the limitation is that the client code is required to control the data members directly, which limits how *currentFont* can be used.

If you program this way, you're likely to have similar lines in many places in your program.

Benefits of Using ADTs

The problem isn't that the ad hoc approach is bad programming practice. It's that you can replace the approach with a better programming practice that produces these benefits:

You can hide implementation details Hiding information about the font data type means that if the data type changes, you can change it in one place without affecting the whole program. For example, unless you hid the implementation details in an ADT, changing the data type from the first representation of bold to the second would entail changing your program in every place in which bold was set rather than in just one place. Hiding the information also protects the rest of the program if you decide to store data in external storage rather than in memory or to rewrite all the font-manipulation routines in another language.

Changes don't affect the whole program If fonts need to become richer and support more operations (such as switching to small caps, superscripts, strikethrough, and so on), you can change the program in one place. The change won't affect the rest of the program.

You can make the interface more informative Code like `currentFont.size = 16` is ambiguous because `16` could be a size in either pixels or points. The context doesn't tell you which is which. Collecting all similar operations into an ADT allows you to define the entire interface in terms of points, or in terms of pixels, or to clearly differentiate between the two, which helps avoid confusing them.

It's easier to improve performance If you need to improve font performance, you can recode a few well-defined routines rather than wading through an entire program.

The program is more obviously correct You can replace the more tedious task of verifying that statements like `currentFont.attribute = currentFont.attribute or 0x02` are correct with the easier task of verifying that calls to `currentFont.SetBoldOn()` are correct. With the first statement, you can have the wrong structure name, the wrong field name, the wrong operation (*and* instead of *or*), or the wrong value for the attribute (`0x20` instead of `0x02`). In the second case, the only thing that could possibly be wrong with the call to `currentFont.SetBoldOn()` is that it's a call to the wrong routine name, so it's easier to see whether it's correct.

The program becomes more self-documenting You can improve statements like `currentFont.attribute or 0x02` by replacing `0x02` with `BOLD` or whatever `0x02` represents, but that doesn't compare to the readability of a routine call such as `currentFont.SetBoldOn()`.



Woodfield, Dunsmore, and Shen conducted a study in which graduate and senior undergraduate computer-science students answered questions about two programs: one that was divided into eight routines along functional lines, and one that was divided into eight abstract-data-type routines (1981). Students using the abstract-data-type program scored over 30 percent higher than students using the functional version.

You don't have to pass data all over your program In the examples just presented, you have to change `currentFont` directly or pass it to every routine that works with fonts. If you use an abstract data type, you don't have to pass `currentFont` all over the program and you don't have to turn it into global data either. The ADT has a structure that contains `currentFont`'s data. The data is directly accessed only by routines that are part of the ADT. Routines that aren't part of the ADT don't have to worry about the data.

You're able to work with real-world entities rather than with low-level implementation structures You can define operations dealing with fonts so that most of the program operates solely in terms of fonts rather than in terms of array accesses, structure definitions, and *True* and *False*.

In this case, to define an abstract data type, you'd define a few routines to control fonts—perhaps like this:

```
currentFont.SetSizeInPoints( sizeInPoints )
currentFont.SetSizeInPixels( sizeInPixels )
currentFont.SetBoldOn()
currentFont.SetBoldOff()
currentFont.SetItalicOn()
currentFont.SetItalicOff()
currentFont.SetTypeFace( faceName )
```



KEY POINT

The code inside these routines would probably be short—it would probably be similar to the code you saw in the ad hoc approach to the font problem earlier. The difference is that you've isolated font operations in a set of routines. That provides a better level of abstraction for the rest of your program to work with fonts, and it gives you a layer of protection against changes in font operations.

More Examples of ADTs

Suppose you're writing software that controls the cooling system for a nuclear reactor. You can treat the cooling system as an abstract data type by defining the following operations for it:

```
coolingSystem.GetTemperature()
coolingSystem.SetCirculationRate( rate )
coolingSystem.OpenValve( valveNumber )
coolingSystem.CloseValve( valveNumber )
```

The specific environment would determine the code written to implement each of these operations. The rest of the program could deal with the cooling system through these functions and wouldn't have to worry about internal details of data-structure implementations, data-structure limitations, changes, and so on.

Here are more examples of abstract data types and likely operations on them:

Cruise Control

- Set speed
- Get current settings
- Resume former speed
- Deactivate

List

- Initialize list
- Insert item in list
- Remove item from list
- Read next item from list

Blender

- Turn on
- Turn off
- Set speed
- Start "Insta-Pulverize"
- Stop "Insta-Pulverize"

Light

- Turn on
- Turn off

Fuel Tank

- Fill tank
- Drain tank
- Get tank capacity
- Get tank status

Stack

- Initialize stack
- Push item onto stack
- Pop item from stack
- Read top of stack

Set of Help Screens	Menu	File
Add help topic	Start new menu	Open file
Remove help topic	Delete menu	Read file
Set current help topic	Add menu item	Write file
Display help screen	Remove menu item	Set current file location
Remove help display	Activate menu item	Close file
Display help index	Deactivate menu item	
Back up to previous screen	Display menu	Elevator
	Hide menu	Move up one floor
Pointer	Get menu choice	Move down one floor
Get pointer to new memory		Move to specific floor
Dispose of memory from existing pointer		Report current floor
Change amount of memory allocated		Return to home floor

You can derive several guidelines from a study of these examples; those guidelines are described in the following subsections:

Build or use typical low-level data types as ADTs, not as low-level data types Most discussions of ADTs focus on representing typical low-level data types as ADTs. As you can see from the examples, you can represent a stack, a list, and a queue, as well as virtually any other typical data type, as an ADT.

The question you need to ask is, “What does this stack, list, or queue represent?” If a stack represents a set of employees, treat the ADT as employees rather than as a stack. If a list represents a set of billing records, treat it as billing records rather than a list. If a queue represents cells in a spreadsheet, treat it as a collection of cells rather than a generic item in a queue. Treat yourself to the highest possible level of abstraction.

Treat common objects such as files as ADTs Most languages include a few abstract data types that you’re probably familiar with but might not think of as ADTs. File operations are a good example. While writing to disk, the operating system spares you the grief of positioning the read/write head at a specific physical address, allocating a new disk sector when you exhaust an old one, and interpreting cryptic error codes. The operating system provides a first level of abstraction and the ADTs for that level. High-level languages provide a second level of abstraction and ADTs for that higher level. A high-level language protects you from the messy details of generating operating-system calls and manipulating data buffers. It allows you to treat a chunk of disk space as a “file.”

You can layer ADTs similarly. If you want to use an ADT at one level that offers data-structure level operations (like pushing and popping a stack), that’s fine. You can create another level on top of that one that works at the level of the real-world problem.

Treat even simple items as ADTs You don't have to have a formidable data type to justify using an abstract data type. One of the ADTs in the example list is a light that supports only two operations—turning it on and turning it off. You might think that it would be a waste to isolate simple “on” and “off” operations in routines of their own, but even simple operations can benefit from the use of ADTs. Putting the light and its operations into an ADT makes the code more self-documenting and easier to change, confines the potential consequences of changes to the *TurnLightOn()* and *TurnLightOff()* routines, and reduces the number of data items you have to pass around.

Refer to an ADT independently of the medium it's stored on Suppose you have an insurance-rates table that's so big that it's always stored on disk. You might be tempted to refer to it as a “rate file” and create access routines such as *RateFile.Read()*. When you refer to it as a file, however, you're exposing more information about the data than you need to. If you ever change the program so that the table is in memory instead of on disk, the code that refers to it as a file will be incorrect, misleading, and confusing. Try to make the names of classes and access routines independent of how the data is stored, and refer to the abstract data type, like the insurance-rates table, instead. That would give your class and access routine names like *rateTable.Read()* or simply *rates.Read()*.

Handling Multiple Instances of Data with ADTs in Non-Object-Oriented Environments

Object-oriented languages provide automatic support for handling multiple instances of an ADT. If you've worked exclusively in object-oriented environments and you've never had to handle the implementation details of multiple instances yourself, count your blessings! (You can also move on to the next section, “ADTs and Classes.”)

If you're working in a non-object-oriented environment such as C, you will have to build support for multiple instances manually. In general, that means including services for the ADT to create and delete instances and designing the ADT's other services so that they can work with multiple instances.

The font ADT originally offered these services:

```
currentFont.SetSize( sizeInPoints )
currentFont.SetBoldOn()
currentFont.SetBoldOff()
currentFont.SetItalicOn()
currentFont.SetItalicOff()
currentFont.SetTypeFace( faceName )
```

In a non-object-oriented environment, these functions would not be attached to a class and would look more like this:

```
SetCurrentFontSize( sizeInPoints )
SetCurrentFontBoldOn()
SetCurrentFontBoldOff()
SetCurrentFontItalicOn()
SetCurrentFontItalicOff()
SetCurrentFontTypeFace( faceName )
```

If you want to work with more than one font at a time, you'll need to add services to create and delete font instances—maybe these:

```
CreateFont( fontId )
DeleteFont( fontId )
SetCurrentFont( fontId )
```

The notion of a *fontId* has been added as a way to keep track of multiple fonts as they're created and used. For other operations, you can choose from among three ways to handle the ADT interface:

- Option 1: Explicitly identify instances each time you use ADT services. In this case, you don't have the notion of a "current font." You pass *fontId* to each routine that manipulates fonts. The *Font* functions keep track of any underlying data, and the client code needs to keep track only of the *fontId*. This requires adding *fontId* as a parameter to each font routine.
- Option 2: Explicitly provide the data used by the ADT services. In this approach, you declare the data that the ADT uses within each routine that uses an ADT service. In other words, you create a *Font* data type that you pass to each of the ADT service routines. You must design the ADT service routines so that they use the *Font* data that's passed to them each time they're called. The client code doesn't need a font ID if you use this approach because it keeps track of the font data itself. (Even though the data is available directly from the *Font* data type, you should access it only with the ADT service routines. This is called keeping the structure "closed.")

The advantage of this approach is that the ADT service routines don't have to look up font information based on a font ID. The disadvantage is that it exposes font data to the rest of the program, which increases the likelihood that client code will make use of the ADT's implementation details that should have remained hidden within the ADT.

- Option 3: Use implicit instances (with great care). Design a new service to call to make a specific font instance the current one—something like *SetCurrentFont(fontId)*. Setting the current font makes all other services use the current font when they're called. If you use this approach, you don't need *fontId* as a parameter to the other services. For simple applications, this can streamline use of

multiple instances. For complex applications, this systemwide dependence on state means that you must keep track of the current font instance throughout code that uses the *Font* functions. Complexity tends to proliferate, and for applications of any size, better alternatives exist.

Inside the abstract data type, you'll have a wealth of options for handling multiple instances, but outside, this sums up the choices if you're working in a non-object-oriented language.

ADTs and Classes

Abstract data types form the foundation for the concept of classes. In languages that support classes, you can implement each abstract data type as its own class. Classes usually involve the additional concepts of inheritance and polymorphism. One way of thinking of a class is as an abstract data type plus inheritance and polymorphism.

6.2 Good Class Interfaces

The first and probably most important step in creating a high-quality class is creating a good interface. This consists of creating a good abstraction for the interface to represent and ensuring that the details remain hidden behind the abstraction.

Good Abstraction

As “Form Consistent Abstractions” in Section 5.3 described, abstraction is the ability to view a complex operation in a simplified form. A class interface provides an abstraction of the implementation that's hidden behind the interface. The class's interface should offer a group of routines that clearly belong together.

You might have a class that implements an employee. It would contain data describing the employee's name, address, phone number, and so on. It would offer services to initialize and use an employee. Here's how that might look.

Cross-Reference Code samples in this book are formatted using a coding convention that emphasizes similarity of styles across multiple languages. For details on the convention (and discussions about multiple coding styles), see “Mixed-Language Programming Considerations” in Section 11.4.

C++ Example of a Class Interface That Presents a Good Abstraction

```
class Employee {
public:
    // public constructors and destructors
    Employee();
    Employee(
        FullName name,
        String address,
        String workPhone,
        String homePhone,
        TaxId taxIdNumber,
        JobClassification jobClass
    );
    virtual ~Employee();
```